



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Stefano Nones
January 2nd, 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix



Executive Summary

The new space race is a global competition, led by the US (NASA/private sector) and China, focusing on returning to the Moon and Mars, with SpaceX as a dominant US player using its reusable rockets.

SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars; other providers cost upward of 165 million dollars each, much of the savings is due to the fact that SpaceX can reuse the first stage.

By using a Data Science approach, I aim to determine the factors supporting the critical first stage landing, with the future perspective to determine the cost of a launch.

To do this, the following operations were performed: data collection, data wrangling, EDA (Exploratory Data Analysis) with data visualization, EDA with SQL, building an interactive map with Folium, building a dashboard with Plotly Dash, and Predictive analysis (Classification).

The EDA analysis indicated that average launch success rate increased yearly (2013-2020). Moreover, the interactive analysis highlighted that KSC LC-39A is the site with the highest number (41.7%) of successful launches and that specific payloads are crucial. Furthermore, the predictive analysis suggested the models to use (LR, SVM, KNN).

Introduction

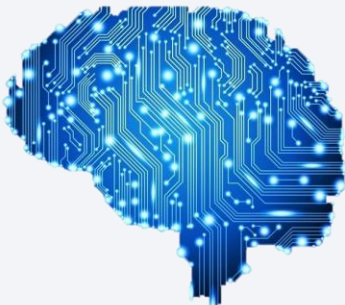
- **Project background and context**



©GW ResearchMagazine

- **SpaceX** leading with reusable rockets like Falcon 9/Heavy
- 62 M USD launch cost (Falcon 9 rocket) Vs. 165 M USD (Others)
- Facing rivals in tourism and emerging players
- Key differentiator → reusability of the first stage

- **Objective**



- Determine if the first stage will land and which factors are involved using Data Science tools
 - Successfully bid against SpaceX for a rocket launch

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - Method 1: Request to the SpaceX API
 - Method 2: Web Scraping from a Wikipedia HTML table
- Perform data wrangling
 - Numeric missing values replaced by mean values
 - Transforming categorical data into a label with one-hot encoding for training supervised models
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - Data were standardized, split in training and testing sets, models were selected (Logistic Regression, Support Vector Machine, Decision Tree classification, K Nearest Neighbors) and the accuracy tested

Data Collection

SpaceX API

- Four helper functions for the extraction of API data were made: 'getBoosterVersion', 'getLaunchSite', 'getPayloadData', 'getCoreData'
- SpaceX launch data were requested and parsed using the GET request
- Missing values in PayloadMass column were replaced by its mean value
- The data frame was filtered to only include Falcon 9 launches

Web Scraping

- Five helper functions to process Web scraped HTML table: 'date_time', 'booster_version', 'landing_status', 'getmass', 'extract_column_from_header'
- The Falcon9 Launch Wiki page was requested from its URL
- All variable names were extracted from the HTML table header
- A data frame was created by parsing the launch HTML tables

Data Collection – SpaceX API

Requested and parsed the data from SpaceX API using GET request



Stored the data and constructed the dataset into a new dictionary with relative columns



Filtered the dataframe to only include Falcon 9 launches needed for the project analysis



Cleaned the data and substituted missing values of PayloadMass

```
[12]: static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.com/resources/fcf_courses/data SpaceX API'

13]: response=requests.get(static_json_url)
```

```
data = response.json()
```

```
df = json_normalize(data)
```

```
[30]: launch_dict = {'FlightNumber': list(data['flight_number']),
                    'Date': list(data['date']),
                    [...]
```

```
# Create a data from launch_dict
```

```
df1 = pd.DataFrame(launch_dict)
```

```
[43]: data_falcon9 = df1[df1['BoosterVersion'] != 'Falcon 1'].copy()
```

```
mean_payload = data_falcon9['PayloadMass'].mean()
data_falcon9.loc[:, 'PayloadMass'] = data_falcon9['PayloadMass'].fillna(mean_payload)
```

Data Collection - Scraping

Requested data from Wikipedia using HTTP GET request and BeautifulSoup



Extracted all column/variable names from the HTML table header



Removed irrelevant columns, created an empty dictionary with extracted column, appended the column names, and filled it up with parsed launch record



Dataframe created

```
[6]: static_url = "https://en.wikipedia.org/w/index.php?title=List
      response = requests.get(static_url, headers=headers)
      soup = BeautifulSoup(response.text, "html.parser")
```

```
      html_tables = soup.find_all("table")
```

```
      first_launch_table = html_tables[2]
```

```
      column_names = []
```

```
      header_cells = first_launch_table.find_all("th")
```

```
      for th in header_cells:
```

```
          name = extract_column_from_header(th)
```

```
          # Only add non-empty names
```

```
          if name is not None and len(name) > 0:
```

```
              column_names.append(name)
```

```
[15]: launch_dict = dict.fromkeys(column_names)
```

```
      # Remove an irrelevant column
```

```
      del launch_dict['Date and time ( )']
```

```
      # Let's initial the launch_dict with each value to be an empty list
```

```
      launch_dict['Flight No.'] = []
```

```
[18]: extracted_row = 0
```

```
      #Extract each table
```

```
      for table_number, table in enumerate(soup.find_all('table')):
```

```
[20]: df = pd.DataFrame({ key:pd.Series(value,dtype="object")
```

Data Wrangling

- Data inspection to determine the variables type composition and the presence of missing values



- Calculation of the number of launches on each site



- Calculation of the number and occurrence of each orbit



- Calculation of the number and occurrence of mission outcome of the orbits



- Creation of a landing outcome label (one-hot encoding: 1, successful landing; 0, unsuccessful) from Outcome column, and of the success rate

```
df.isnull().sum()/len(df)*100
```

```
df.dtypes
```

```
df['LaunchSite'].value_counts()
```

```
df[df['Orbit'] != 'GTO']['Orbit'].value_counts()
```

```
landing_outcomes=df['Outcome'].value_counts()  
landing_outcomes
```

```
for i,outcome in enumerate(landing_outcomes.keys()):  
    print(i,outcome)
```

```
bad_outcomes=set(landing_outcomes.keys()[[1,3,5,6,7]])  
bad_outcomes
```

```
landing_class = [0 if outcome in bad_outcomes else 1  
                 for outcome in df['Outcome']]
```

```
df['Class']=landing_class  
df["Class"].mean()
```

EDA with Data Visualization

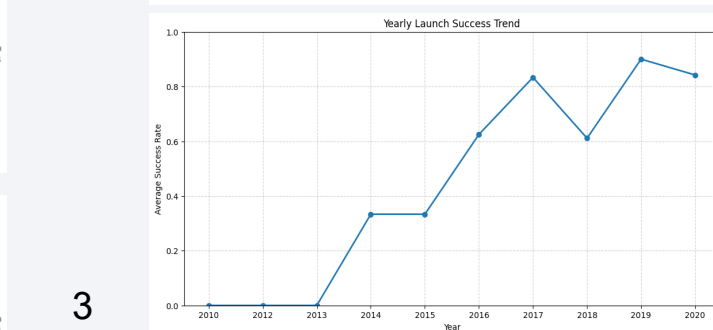
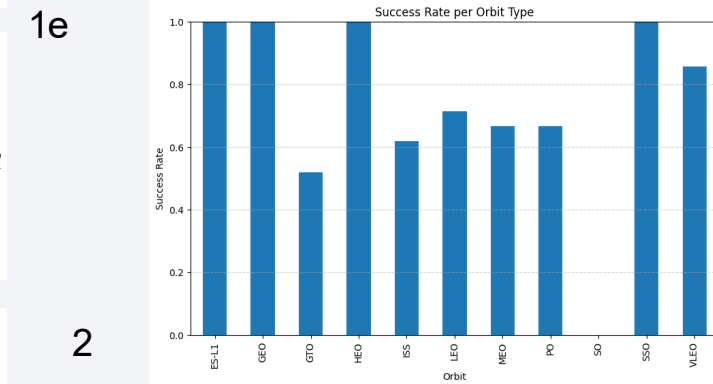
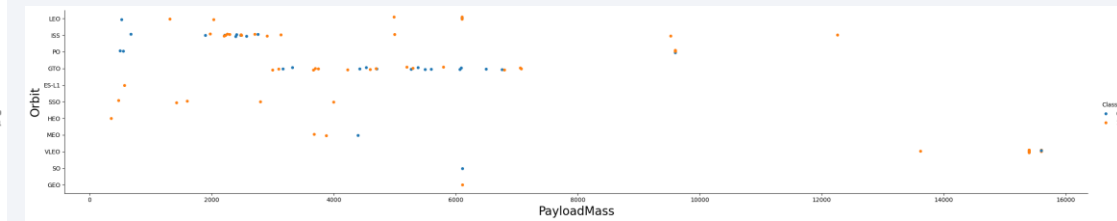
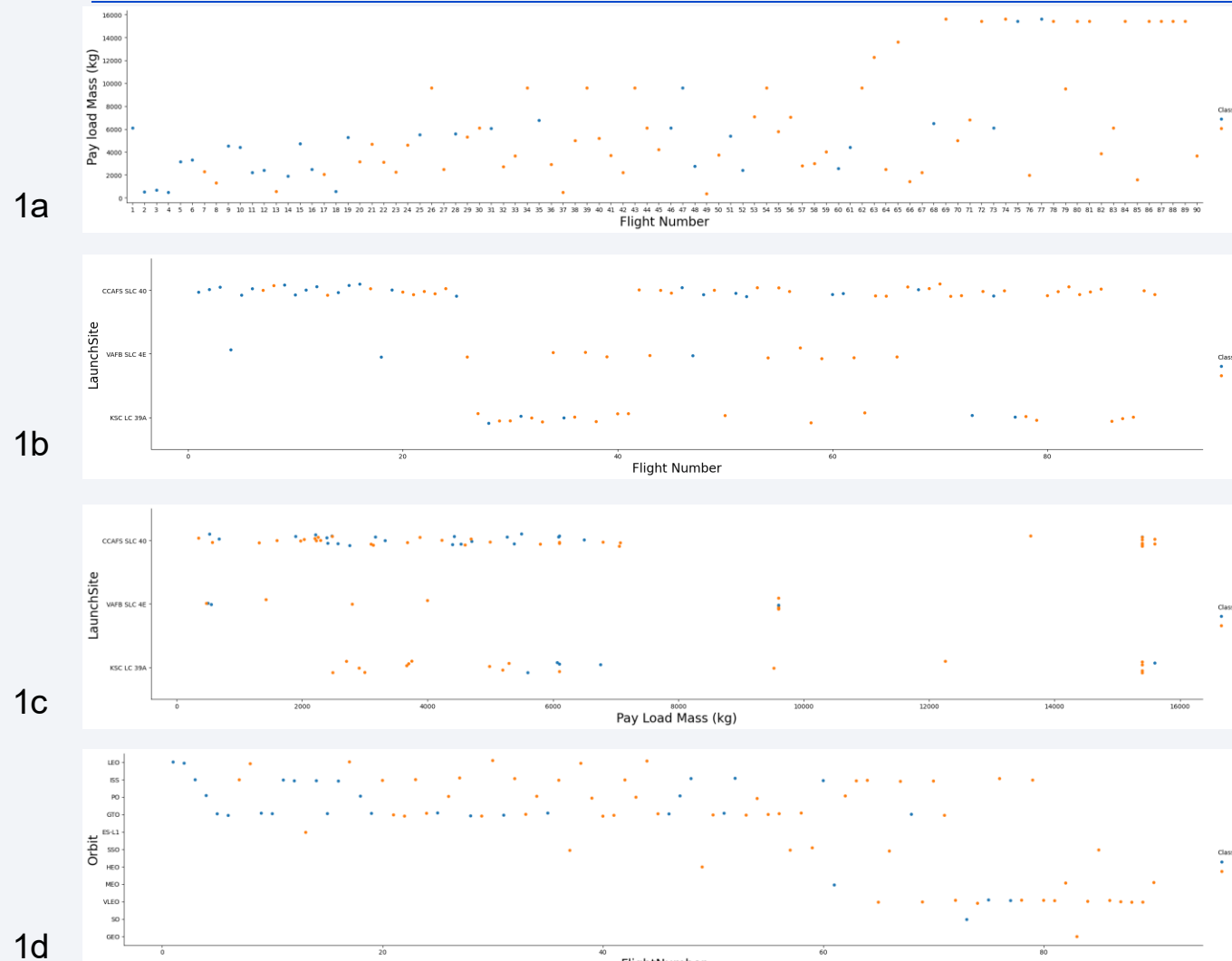


Figure 2. Bar chart visualizing the relationship between success rate of each orbit type;

Figure 3. Line chart visualizing the launch success yearly trend

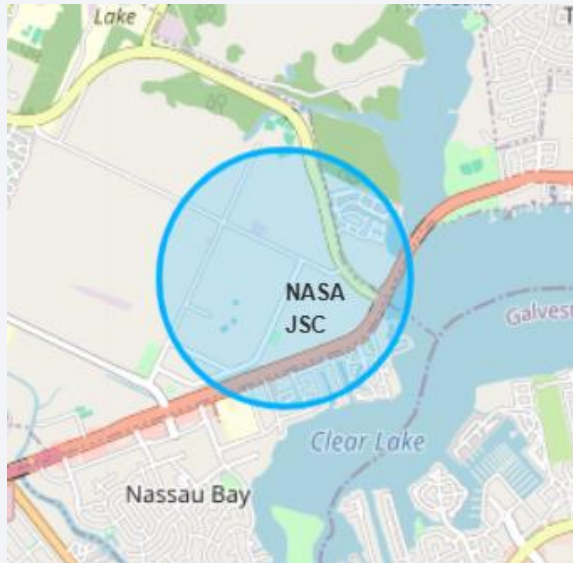
Figure 1. Scatterplots are used to visualize the relationship between: **a.** Flight number (FN) and Payload mass (PM), **b.** FN and Launch Site, **c.** PM and Launch Site, **d.** FN and Orbit type, **e.** PM and Orbit type

EDA with SQL

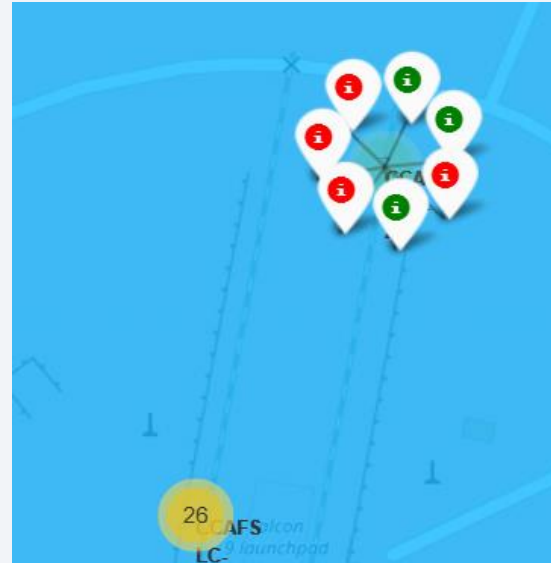
1. Unique launch sites
2. Launch sites beginning with 'CCA' (5)
3. Total payload mass by boosters from NASA (CRS)
4. Average payload mass by booster F9 v1.1
5. Date first successful landing outcome in ground pad
6. Boosters drone ship success ($4000 < \text{payload mass} < 6000$)
7. Total successful and failure mission outcomes
8. Booster_versions with maximum payload mass
9. Month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.
10. Rank count landing outcomes (such as Failure (drone ship) or Success (ground pad)) 2010/06/04 - 2017/03/20, descending order.

Build an Interactive Map with Folium

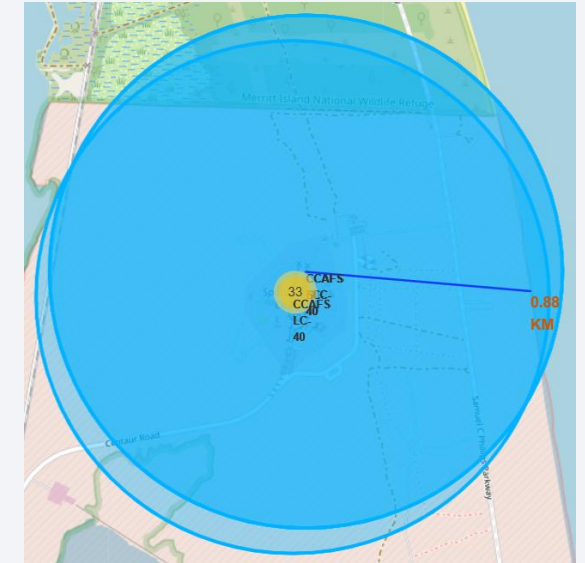
4



5

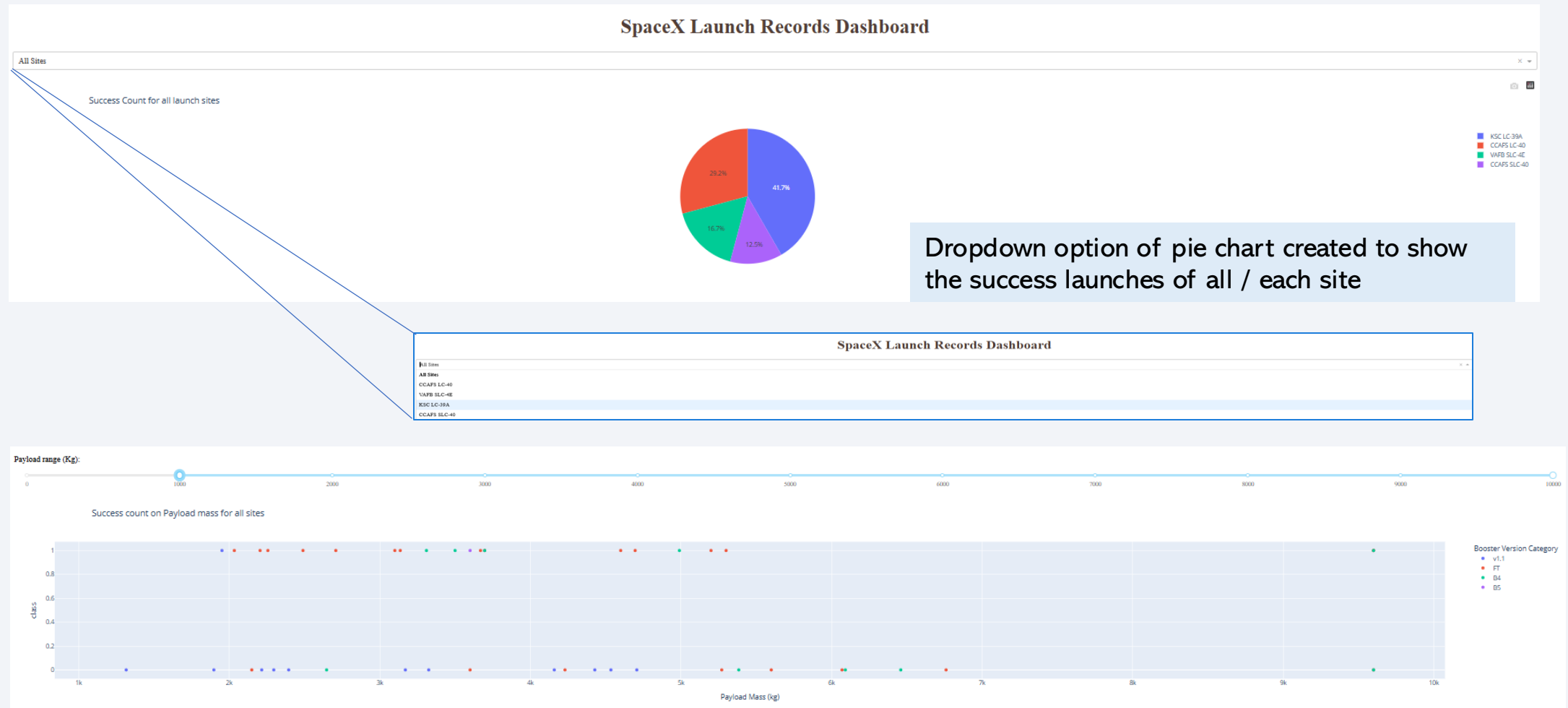


6



- **Figure 4:** Circle area to highlight a specific location (NASA JSC)
- **Figure 5:** Marker cluster with marker colors based on the class value to inform the reader if the launch was successful or failed
- **Figure 6:** Line between a launch site to its closest infrastructure to represent the distance

Build a Dashboard with Plotly Dash



[GitHub Build a Dashboard with Plotly Dash](#)

Scatter plot with Payload mass range slider to show the success launches of each/all sites and booster version category by payload mass

Predictive Analysis (Classification)

```
Y = data['Class'].to_numpy()

[9]: from sklearn import preprocessing
transform = preprocessing.StandardScaler()
X = transform.fit_transform(X)

[10]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)

[12]: parameters = {'C': [0.01, 0.1, 1],
                    'penalty': ['l2'],
                    'solver': ['lbfgs']}

[17]: parameters = {'C': [0.01, 0.1, 1], 'penalty': ['l2'],
                    lr=LogisticRegression()}

# GridSearchCV object
logreg_cv = GridSearchCV(lr, parameters, cv=10)

# Fit to training data
logreg_cv.fit(X_train, Y_train)

[17]: GridSearchCV
       estimator: LogisticRegression
              > LogisticRegression

[19]: test_accuracy = logreg_cv.score(X_test, Y_test)
print("Test accuracy:", test_accuracy)

Test accuracy: 0.8333333333333334

[20]: yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

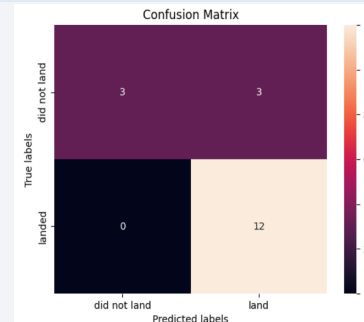
Process example
(Logistic Regression)

LR, SVM, Decision Tree and KNN objects were created and fitted with GridSearchCV object to find the best parameters. Then the models were trained on the training set.

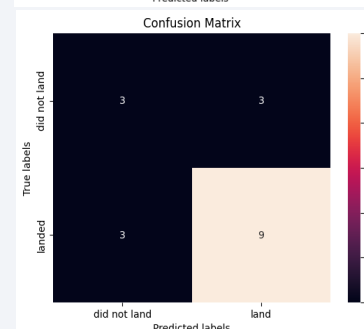


The accuracies of test data were calculated for each machine learning model and the confusion matrix plotted

LR, SVM, KNN



Decision tree



TASK 12

Find the method performs best:

```
[37]: print('LR Accuracy:', '{:.2%}'
print('SVM Accuracy:', '{:.2%}'
print('Decision Tree Accuracy:
print('KNN Accuracy:', '{:.2%}'
#Based on the results from thi
```

LR Accuracy: 83.33%
SVM Accuracy: 83.33%
Decision Tree Accuracy: 66.67%
KNN Accuracy: 83.33%

Results 1

EDA with data visualization

- Average launch success rate increased yearly (2013-2020) and landing is benefitted by continuous launch attempts. Most failures were under 40 number of flights
- Payloads with lower masses had more launches than higher ones
- The landing success for each orbit type seems to be linked to the increasing number of flights, although with exceptions. The orbits with the highest success rates are: ES-L1, GEO, HEO, SSO
- Light payloads had successful landings for four orbits (ES-L1, GEO, SSO, MEO (<4000 Kg)), heaviest payloads were tested for orbit ISS and VLEO, successfully (critical when in proximity of 16000 Kg), two orbits had less predictable success (GTO, PO) and SO none

EDA with SQL

- Four distinct launch sites: CCAFS LC-40, VAFB SLC-4E, KSC LC-39A, CCAFS SLC-40
- Average payload mass carried by the booster F9 v1.1 is 2928.4 Kg
- 100 successful mission outcomes and one failure; F9 ver. FT boosters (4000-6000 Kg) landed on drone ships, ver. B5 boosters carried max payload mass; most landing attempts (2010-2017) were on drone ships

Results 2

Interactive analytics demo in screenshots

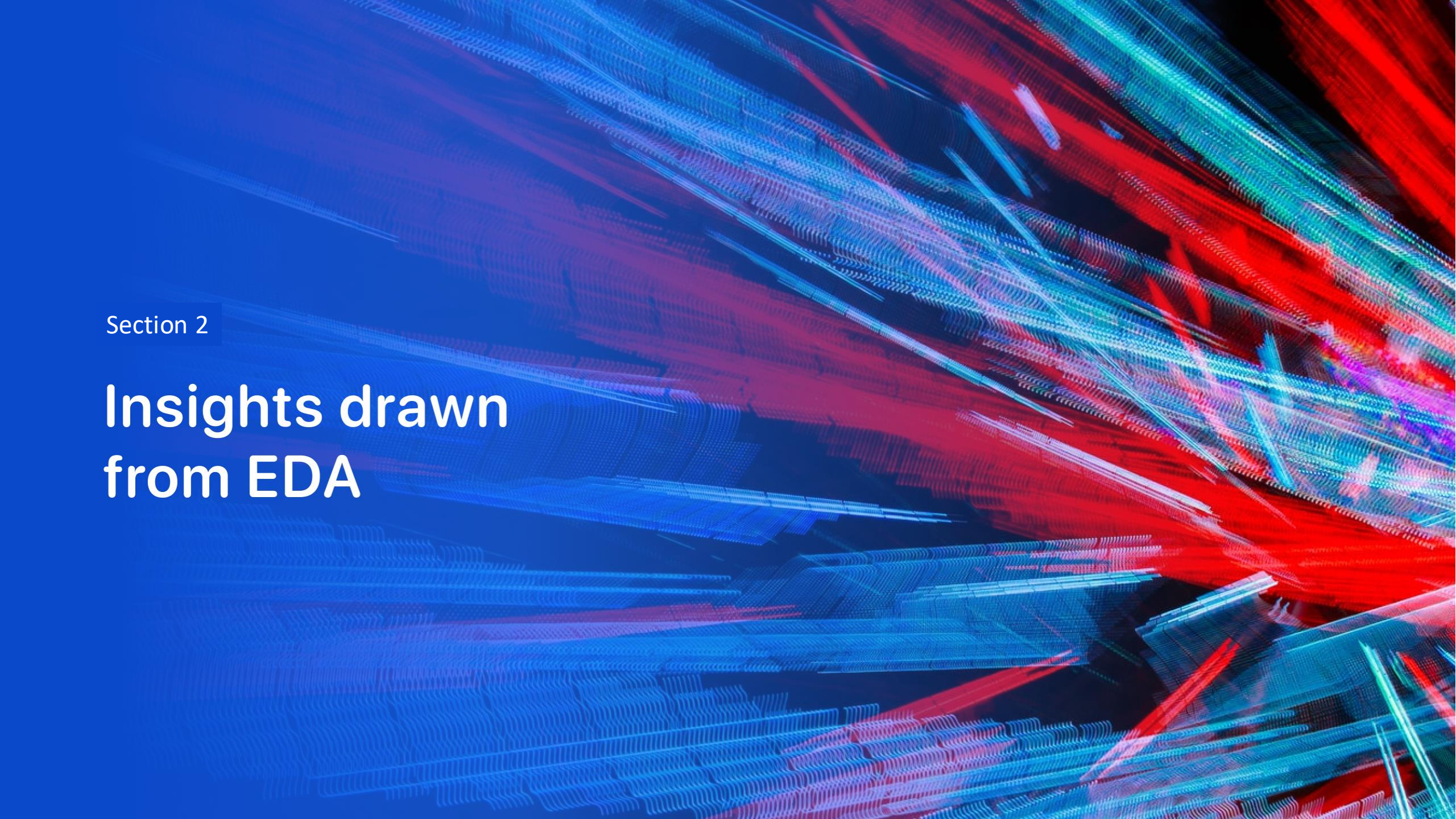
- Launch Complex 39A at Kennedy Space Center (KSC LC-39A) had the highest number (41.7%) of successful launches compared to other launch sites.
- KSC LC-39A had a 76.9% success rate for the launches from its site, and 23.1% failure rate
- The payload range (Kg) for all sites with:
 - the highest launch success rate is 3000-4000 Kg,
 - the lowest launch success rates are in the range of 500-1500 Kg and 5000-7000 Kg
- The F9 booster version with the highest launch success rate is FT



KSC LC-39A site

Predictive analysis results

The models LR, SVM, and KNN achieved a higher test accuracy (83.33%) than the Decision tree (66.67%), predicting all the 12 successful landing correctly, although showing three false positives.

The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of blue and red, creating a sense of motion or data flow. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is high-tech and digital.

Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

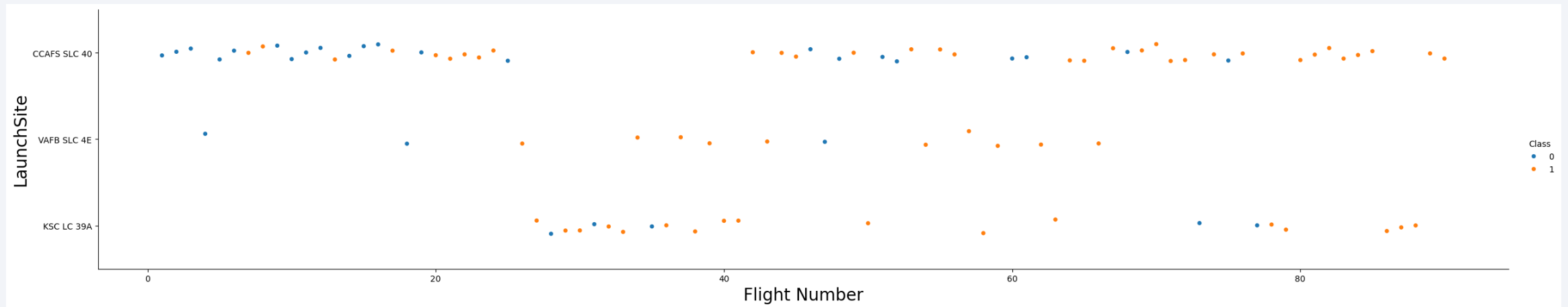


Figure 1b. Scatter plot of Flight Number vs. Launch Site.

The scatter plot suggests that the continuous launch attempts (Flight Number) generally lead to better success for the first stage to land successfully in each Launch Site. In fact, all the launch sites had most of their failures with flight number under 40.

Payload vs. Launch Site

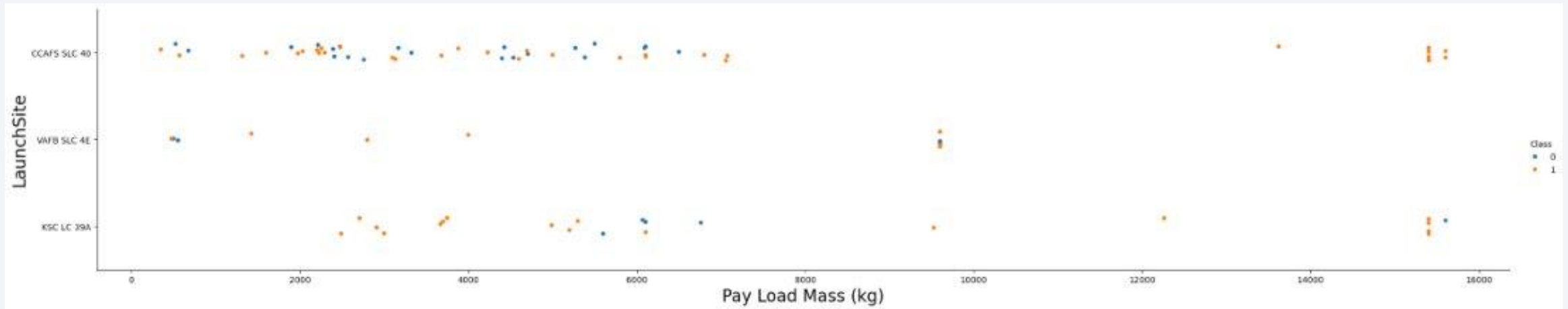


Figure 1c. Scatter plot of Payload vs. Launch Site.

The scatter plot shows that payloads with a lower mass have more launches compared to those with a higher mass across all three launch sites.

Moreover, for the VAFB-SLC launch site there are no rockets launched for heavy payload mass (greater than 10000 Kg).

Success Rate vs. Orbit Type

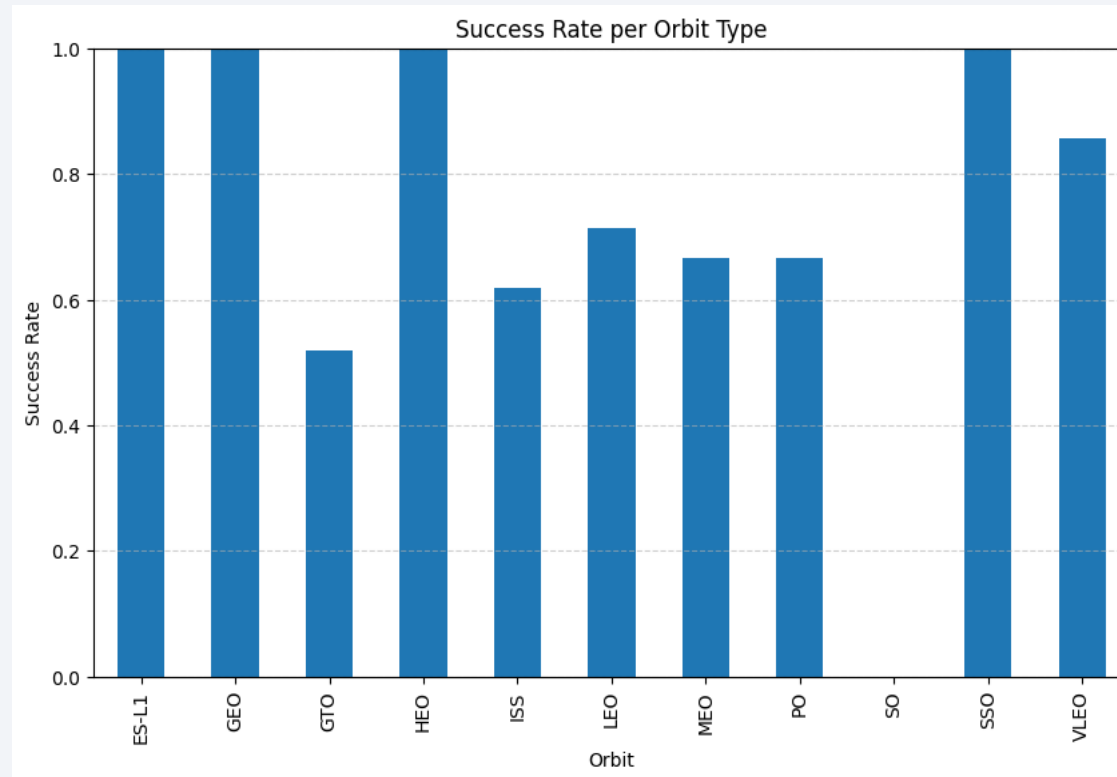


Figure 2. Bar plot of Success Rate vs. Orbit Type. The bar plot shows that the orbits with the highest success rates are: ES-L1, GEO, HEO, SSO.

Flight Number vs. Orbit Type

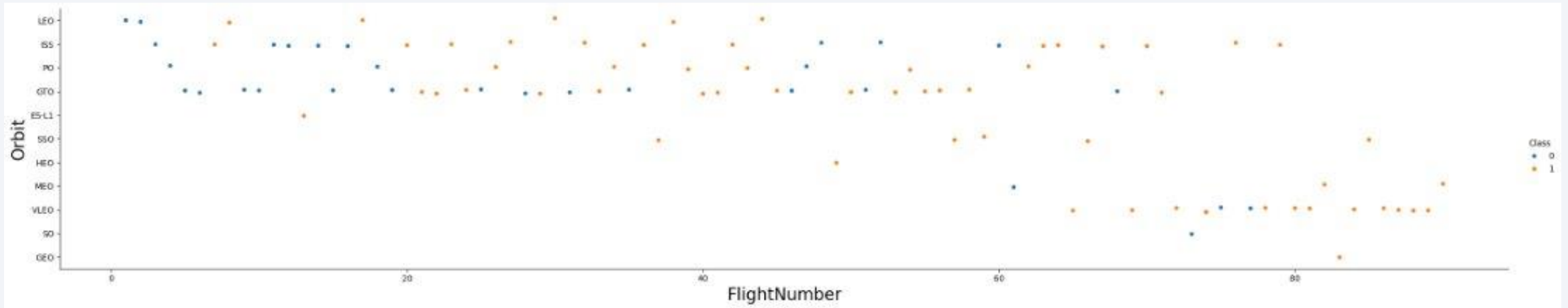


Figure 1d. Scatter plot of Flight Number vs. Orbit Type.

The scatter plot suggests that generally the landing success for each orbit seems to be related to the overall increasing number of flights. This is clearer for the LEO orbit. Conversely, in the GTO orbit, the relationship between flight number and success is less clear and other factors might be involved.

Payload vs. Orbit Type

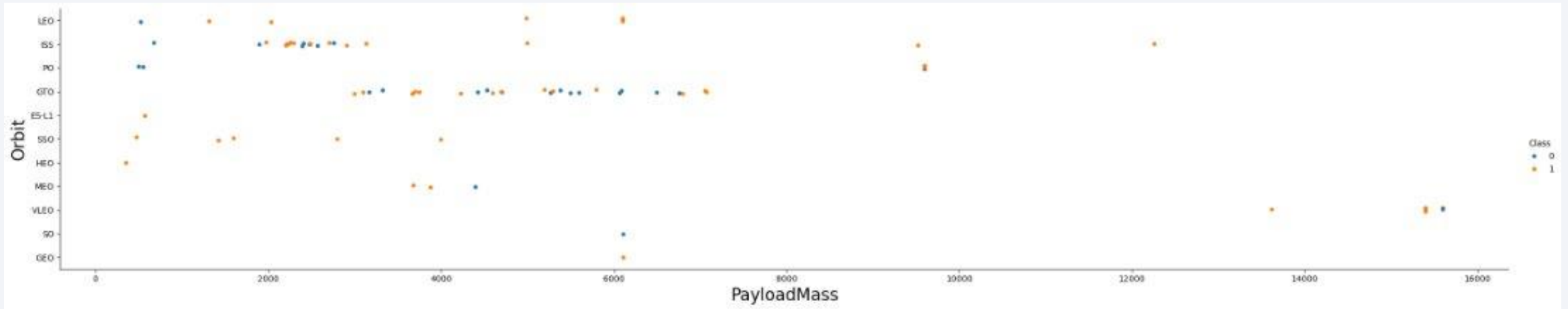


Figure 1e. Scatter plot of Payload vs. Orbit Type.

The scatter plot suggests that increasing the payloads weight might benefit the successful landing or positive landing rate in relation to each orbit type. Moreover, orbits can be divided into three major groups:

- (i) Light payloads with successful landings (ES-L1, SSO, MEO (<4000 Kg), GEO)
- (ii) Heaviest payloads (> 10000 Kg) tested for orbit ISS, giving only successful landings, and VLEO. Although VLEO had successful landing with a record payload of over 15000 Kg, it showed that limits might exist, in proximity of 16000 Kg.
- (iii) Orbits with uncertain landing success: overlap of successes and failures for similar payloads (GTO, PO) ²³

Launch Success Yearly Trend

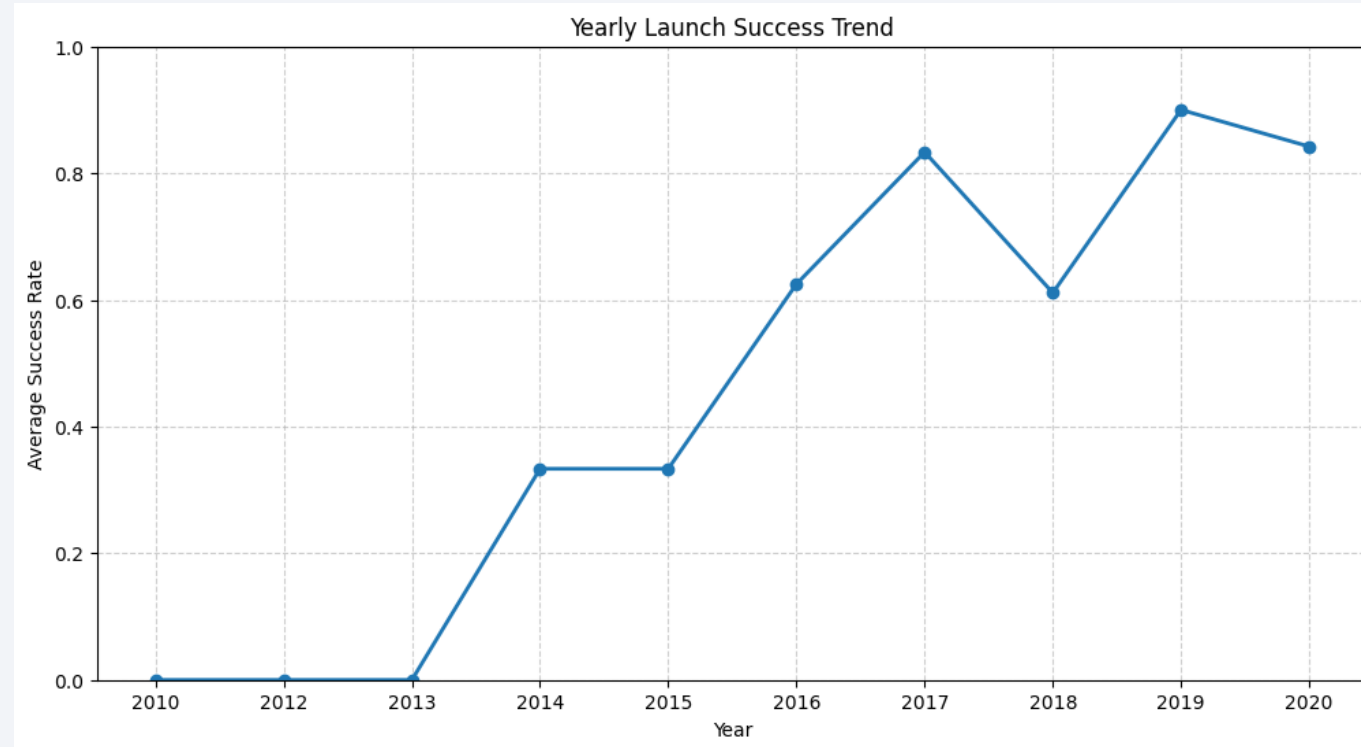


Figure 3. Line chart of yearly launch success trend. The line chart shows that the average success rate since 2013 kept increasing till 2020, possibly due to technology advancement and experience.

All Launch Site Names

Task 1

Display the names of the unique launch sites in the space

```
[12]: %sql SELECT DISTINCT LAUNCH_SITE FROM SPACEXTABLE;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
[12]: Launch_Site
```

```
CCAFS LC-40
```

```
VAFB SLC-4E
```

```
KSC LC-39A
```

```
CCAFS SLC-40
```

- Performed an SQL query to obtain all launch site names

Launch Site Names Begin with 'CCA'

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
[14]: %sql SELECT * FROM SPACEXTABLE WHERE LAUNCH_SITE LIKE 'CCA%' LIMIT 5;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

[14]:	Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
	2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
	2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
	2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
	2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
	2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

- Performed an SQL query to obtain 5 launch site names that begin with 'CCA'

Total Payload Mass

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
[16]: %sql SELECT SUM(PAYLOAD_MASS__KG_) AS total_payload_mass FROM SPACEXTABLE WHERE CUSTOMER = 'NASA (CRS)';
* sqlite:///my_data1.db
Done.
[16]: total_payload_mass
      45596
```

- Performed an SQL query to obtain the total payload mass carried by boosters launched by NASA (CRS)

Average Payload Mass by F9 v1.1

Task 4

Display average payload mass carried by booster version F9 v1.1

```
[17]: %sql SELECT AVG(PAYLOAD_MASS_KG_) FROM SPACEXTABLE WHERE BOOSTER_VERSION = 'F9 v1.1';
```

```
* sqlite:///my_data1.db  
Done.
```

```
[17]: AVG(PAYLOAD_MASS_KG_)  
2928.4
```

- Performed an SQL query to calculate the average payload mass carried by booster version F9 v1.1

First Successful Ground Landing Date

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
[18]: %sql SELECT MIN("DATE") FROM SPACEXTABLE WHERE LANDING_OUTCOME = 'Success (ground pad)';  
* sqlite:///my_data1.db  
Done.  
[18]: MIN("DATE")  
2015-12-22
```

- Performed an SQL query to find the dates of the first successful landing outcome on ground pad

Successful Drone Ship Landing with Payload between 4000 and 6000

```
%sql SELECT Booster_Version FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (drone ship)' AND payload_mass__kg_ > 4000 AND payload_mass__kg_ < 6000;
```

Task 6

List the names of the boosters which have success in dro

```
[19]: %sql SELECT Booster_Version FROM SPACEXTABLE WHERE
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
[19]: Booster_Version
```

```
F9 FT B1022
```

```
F9 FT B1026
```

```
F9 FT B1021.2
```

```
F9 FT B1031.2
```

- Performed an SQL query to list the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000

Total Number of Successful and Failure Mission Outcomes

Task 7

List the total number of successful and failure mission outcomes

```
[20]: %sql SELECT Mission_Outcome, COUNT(*) AS total FROM SPACEXTABLE GROUP BY Mission_Outcome;
```

```
* sqlite:///my_data1.db  
Done.
```

```
[20]:
```

Mission_Outcome	total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

- Performed an SQL query to calculate the total number of successful and failure mission outcomes

Boosters Carried Maximum Payload

```
%sql SELECT Booster_Version FROM SPACEXTABLE WHERE payload_mass__kg_ = ( SELECT MAX(payload_mass__kg_) FROM SPACEXTABLE );
```

Task 8

List all the booster_versions that have carried the maximum payload mass, using the following query:

```
[22]: %sql SELECT Booster_Version FROM SPACEXTABLE WHERE payload_mass__kg_ = ( SELECT MAX(payload_mass__kg_) FROM SPACEXTABLE );
```

```
* sqlite:///my_data1.db  
Done.
```

```
[22]: Booster_Version
```

```
F9 B5 B1048.4
```

```
F9 B5 B1049.4
```

```
F9 B5 B1051.3
```

```
F9 B5 B1056.4
```

```
F9 B5 B1048.5
```

```
F9 B5 B1051.4
```

```
F9 B5 B1049.5
```

```
F9 B5 B1060.2
```

```
F9 B5 B1058.3
```

```
F9 B5 B1051.6
```

```
F9 B5 B1060.3
```

```
F9 B5 B1049.7
```

- Performed an SQL query to list the names of the booster which have carried the maximum payload mass

2015 Launch Records

```
%sql SELECT CASE WHEN substr("DATE", 6, 2) = '01' THEN 'January' WHEN substr("DATE", 6, 2) = '02' THEN 'February' WHEN substr("DATE", 6, 2) = '03' THEN 'March' WHEN substr("DATE", 6, 2) = '04' THEN 'April' WHEN substr("DATE", 6, 2) = '05' THEN 'May' WHEN substr("DATE", 6, 2) = '06' THEN 'June' WHEN substr("DATE", 6, 2) = '07' THEN 'July' WHEN substr("DATE", 6, 2) = '08' THEN 'August' WHEN substr("DATE", 6, 2) = '09' THEN 'September' WHEN substr("DATE", 6, 2) = '10' THEN 'October' WHEN substr("DATE", 6, 2) = '11' THEN 'November' WHEN substr("DATE", 6, 2) = '12' THEN 'December' END AS Month_Name, Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTABLE WHERE Landing_Outcome IN ('Failure (drone ship)', 'Failure (ground pad)', 'Failure (other)') AND substr("DATE", 0, 5) = '2015';
```

Task 9

List the records which will display the month names, failure landing_out

Note: SQLite does not support monthnames. So you need to use su

```
[33]: %sql SELECT CASE WHEN substr("DATE", 6, 2) = '01' THEN 'January'
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
t[33]:
```

Month_Name	Landing_Outcome	Booster_Version	Launch_Site
------------	-----------------	-----------------	-------------

January	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
---------	----------------------	---------------	-------------

April	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40
-------	----------------------	---------------	-------------

- Performed an SQL query to list the failed landing outcomes in drone ship, their booster versions, and launch site names for in year 2015

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

```
%sql SELECT Landing_Outcome, COUNT(*) AS Outcome_Count FROM SPACEXTABLE WHERE "DATE" BETWEEN '2010-06-04' AND '2017-03-20'  
GROUP BY Landing_Outcome ORDER BY Outcome_Count DESC
```

Task 10

Rank the count of landing outcomes (such as Failure (

```
[35]: %sql SELECT Landing_Outcome, COUNT(*) AS Outcome
```

```
* sqlite:///my_data1.db  
Done.
```

```
[35]:
```

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

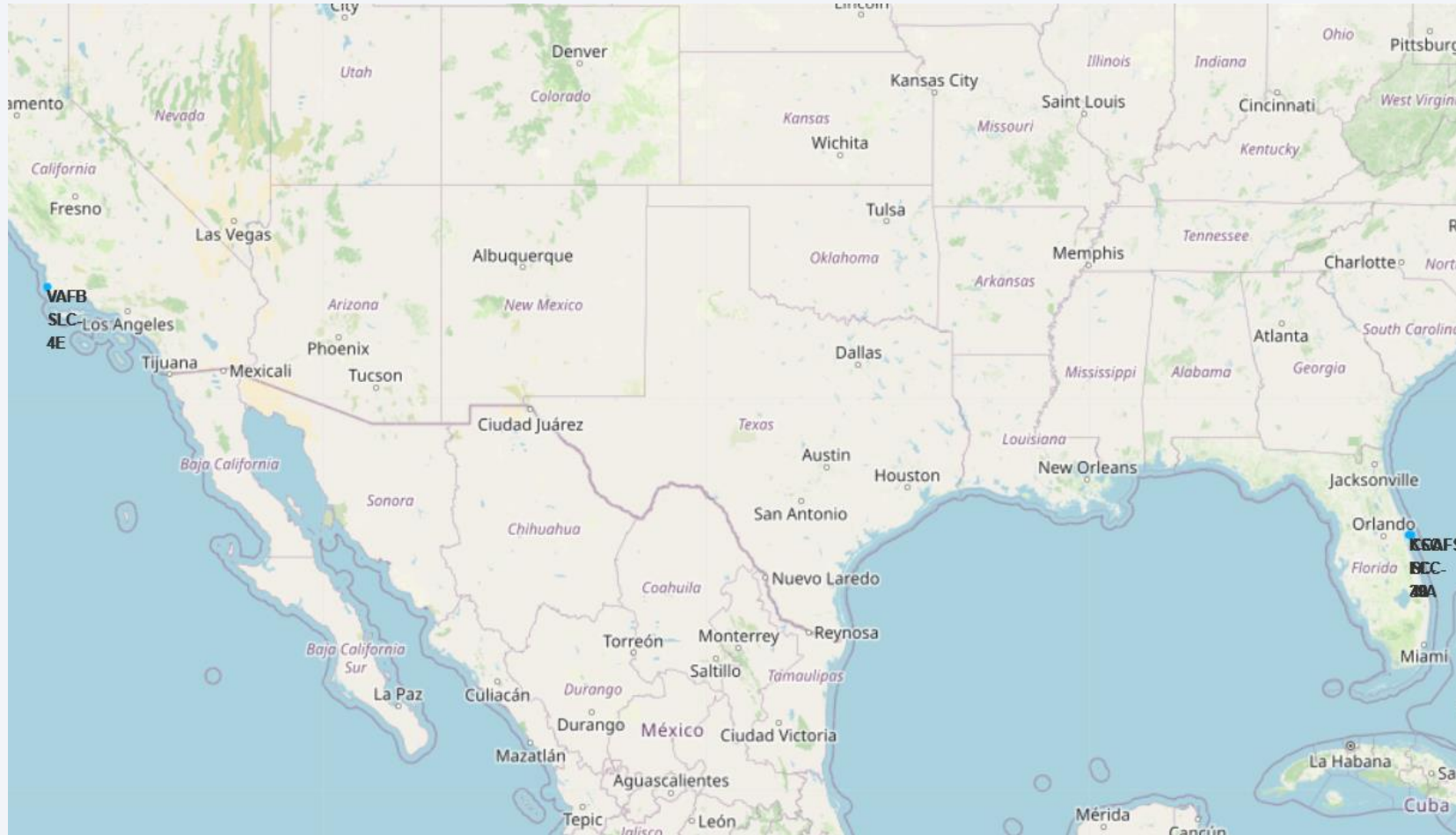
- Performed an SQL query to rank the count of landing outcomes (such as Failure drone ship) or Success ground pad) between the date 2010-06-04 and 2017-03-20, in descending order

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a solid blue background on the left and a satellite photograph of Earth on the right. The Earth's surface is dark, with numerous bright yellow and orange lights representing cities and urban areas. The horizon of the Earth is visible as a thin, curved line separating the dark surface from the deep blue of space.

Section 3

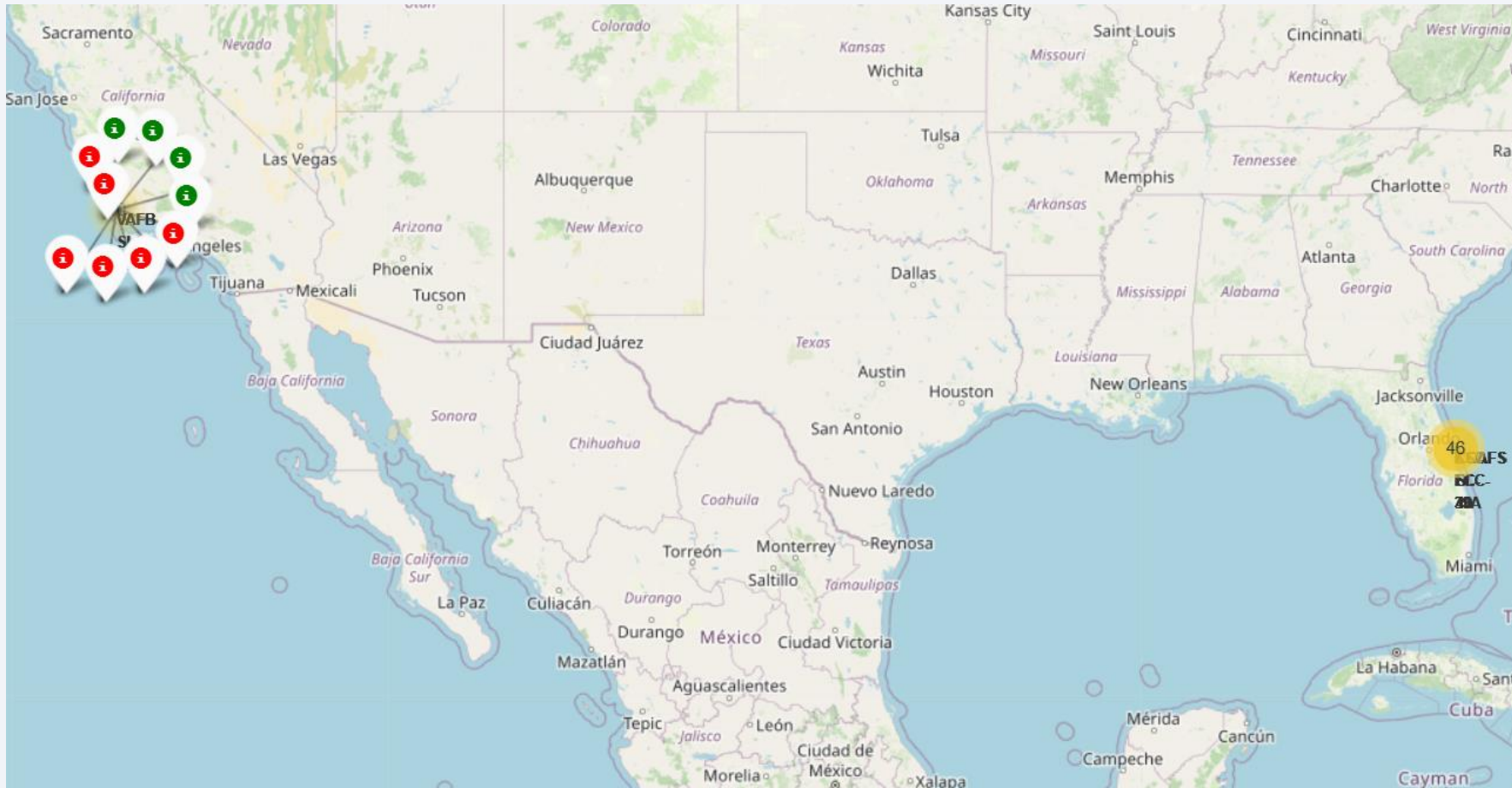
Launch Sites Proximities Analysis

The launch sites across US



- The launch sites are labelled by a blue marker with their names on the map: one on the West Coast (VAFB SLC-4E) and three on the East Coast (CCAFS LC-40, CCAFS SLC-40, KSC LC-39A)

Launch outcomes on the map

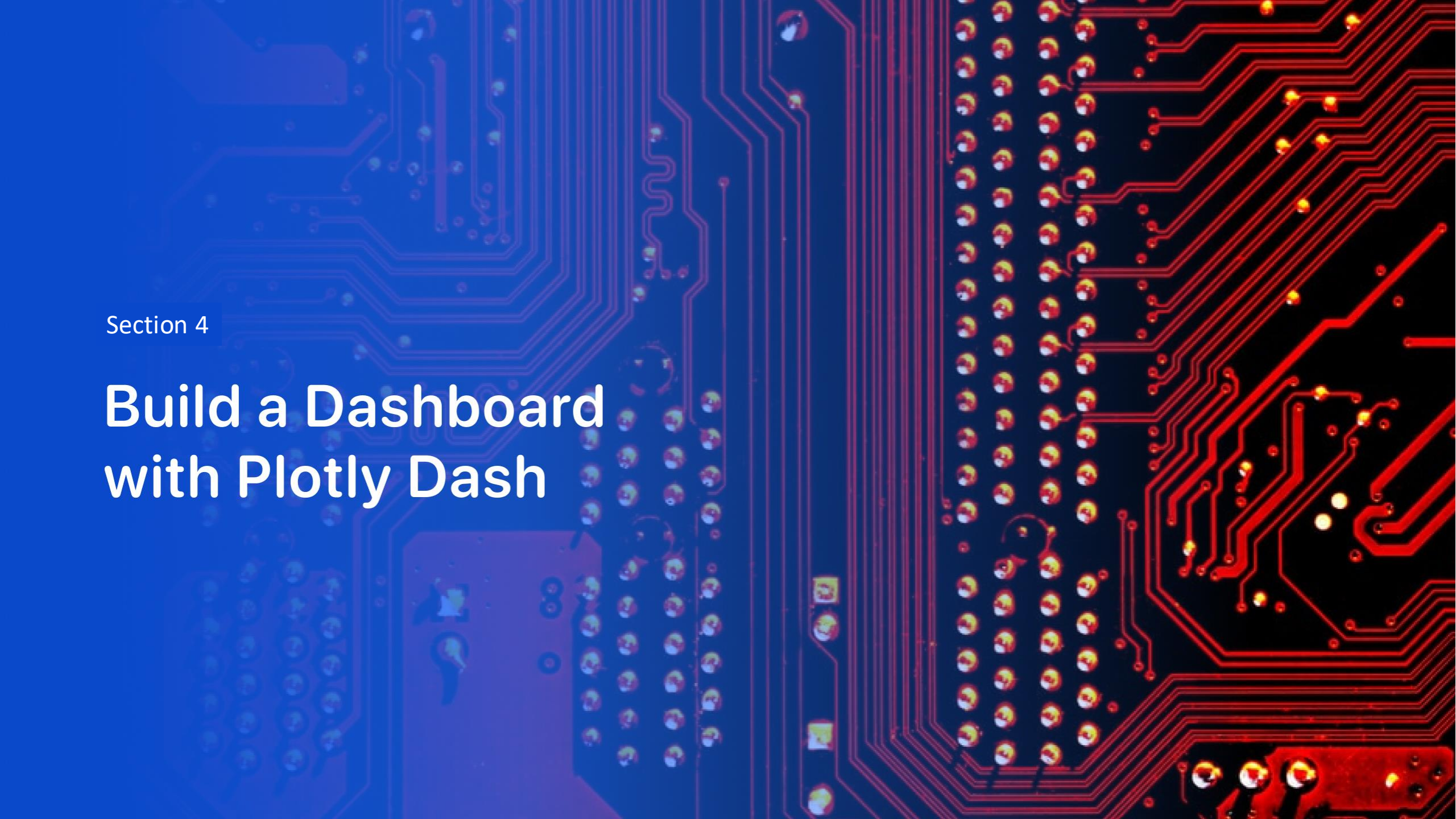


- The launch records are grouped in clusters on the map, then labelled by green markers for successful launches, and red markers for unsuccessful ones.
- The map in the screenshot shows that 4 out of 10 launches were successful in VAFB SLC-4E, and that other 46 launches are mapped for the other three sites

Distances between a launch site to its proximities



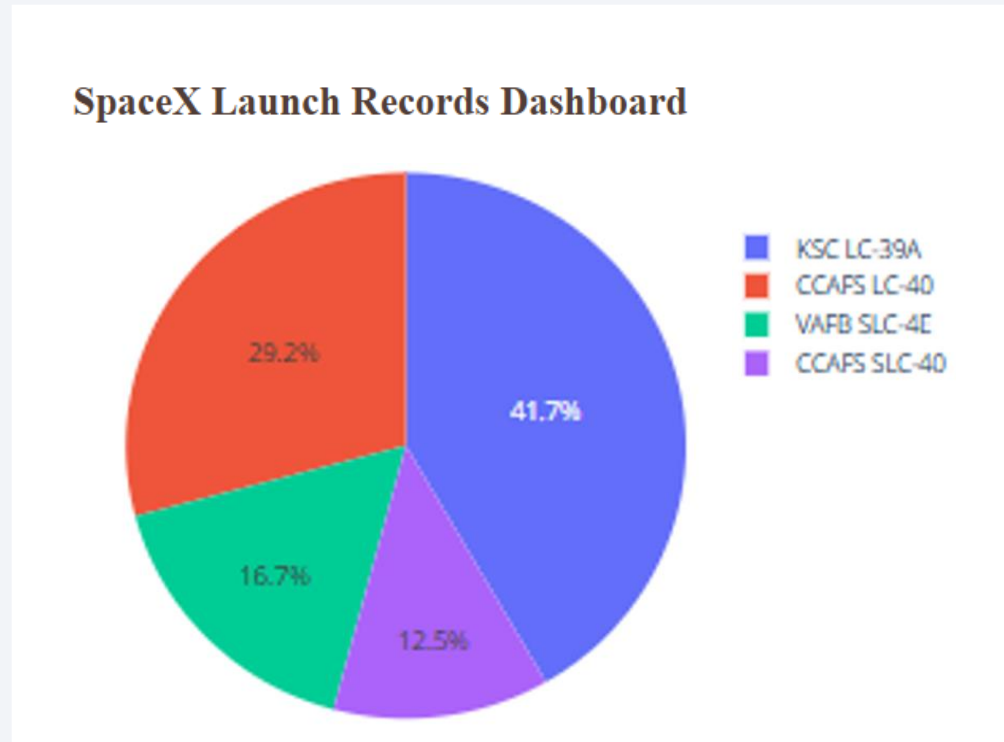
- The closest highway and coastline from the launch site CCAFS SLC-40 are marked as points using MousePosition. The distance between them and the launch site, is approximately of 0.59 and 0.88 Km, respectively.



Section 4

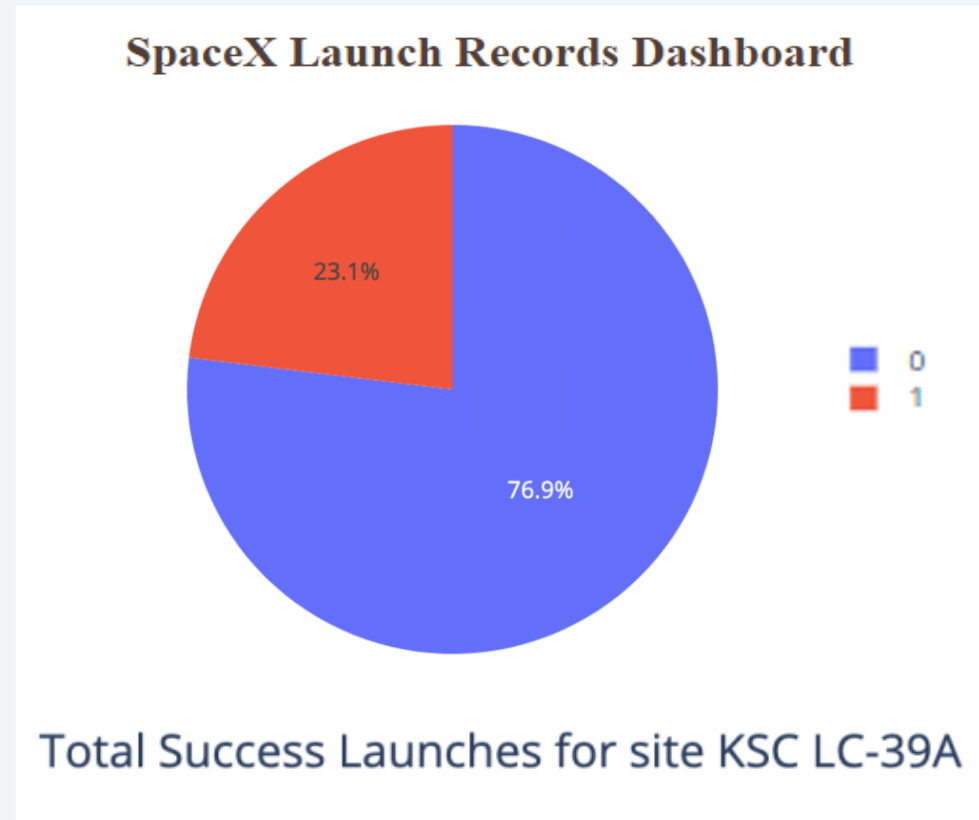
Build a Dashboard with Plotly Dash

Total success launches for each site (%)



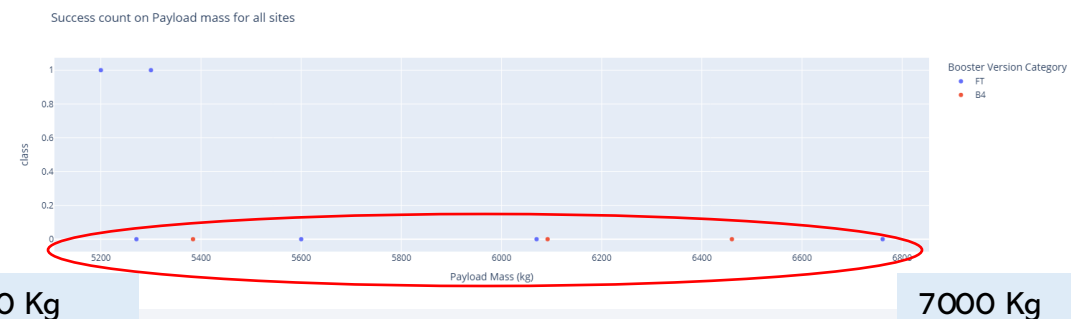
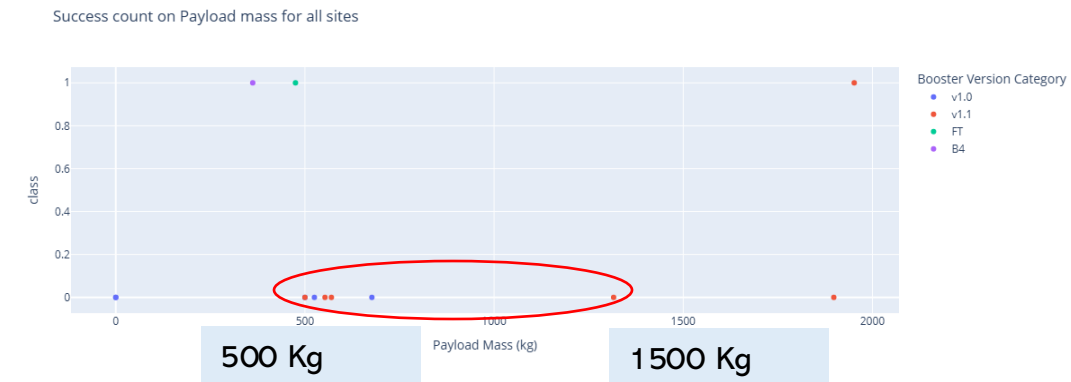
- KSC LC-39A has the highest amount of success launches with 41.7% from the entire record, whereas CCAFS SLC-40 has the lowest amount of success launches with only 12.5%

Total success launches for site KSC LC-39A (%)



- KSC LC-39A which is the launch site with highest amount of success, has a 76.9% success rate for the launches from its site, and 23.1% failure rate

Success count on Payload mass for all sites by booster version



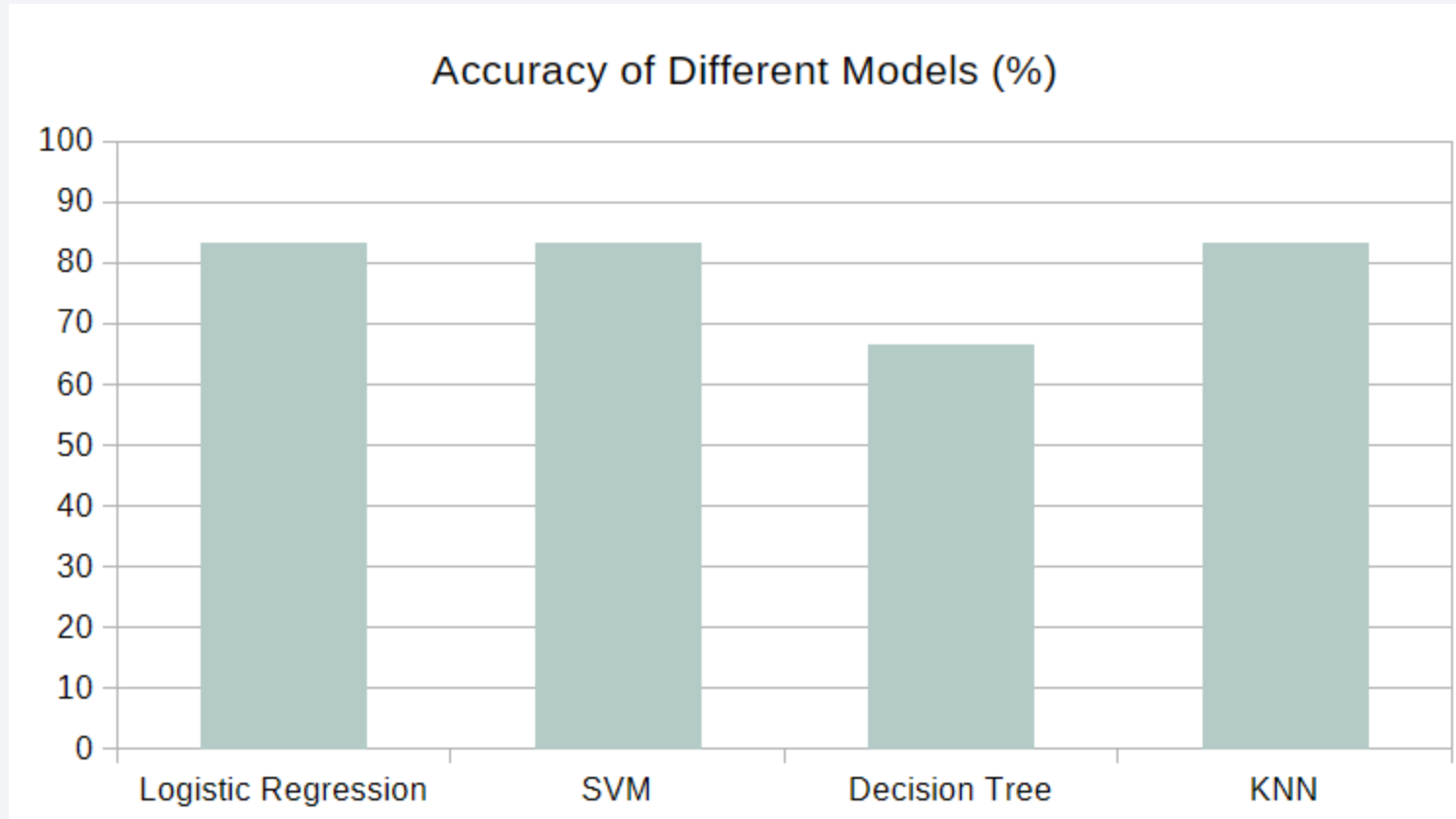
- The payload range (Kg) with the highest launch success rate is that one of 3000-4000 Kg, presenting seven successes and only three failures.
- The payload ranges (Kg) with the lowest launch success rate are those ones of 500-1500 Kg with six failures, and 5-7000 Kg with only two successes (FT) and seven failures.
- The F9 booster version with the highest launch success rate is FT



Section 5

Predictive Analysis (Classification)

Classification Accuracy

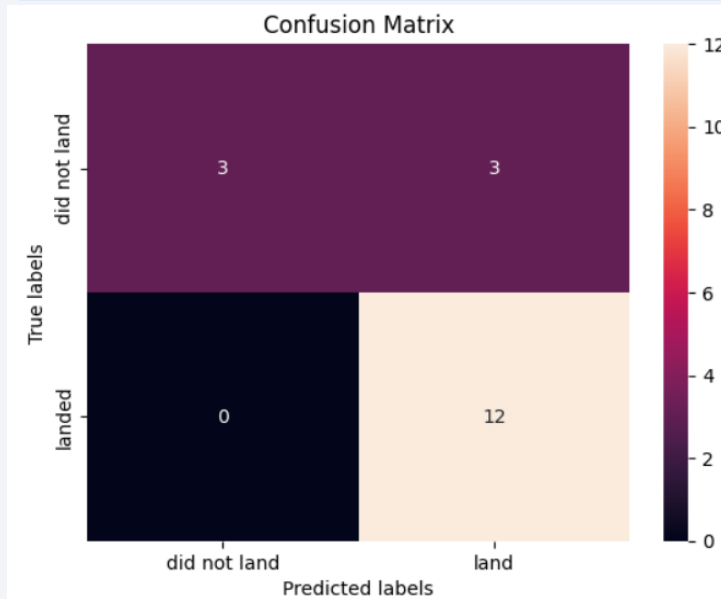


- The models that performed best were LR, SVM, and KNN, all achieving the highest accuracy of 83.33%

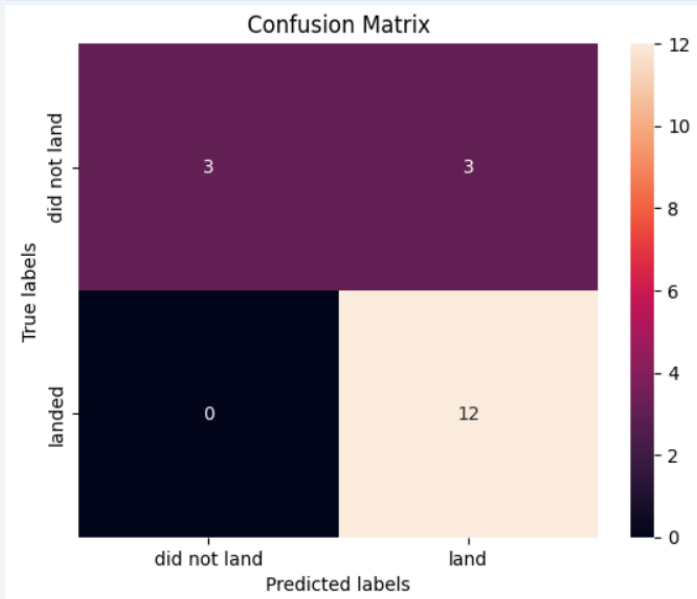
Note on the Decision tree model. I updated the parameters that were originally present in the course presets, because deprecated. The results with the new parameters showed that although the cross-validation accuracy computed on training data was higher (99%), the test accuracy was lower than the test accuracies of the other models (66.67%).

Confusion Matrix

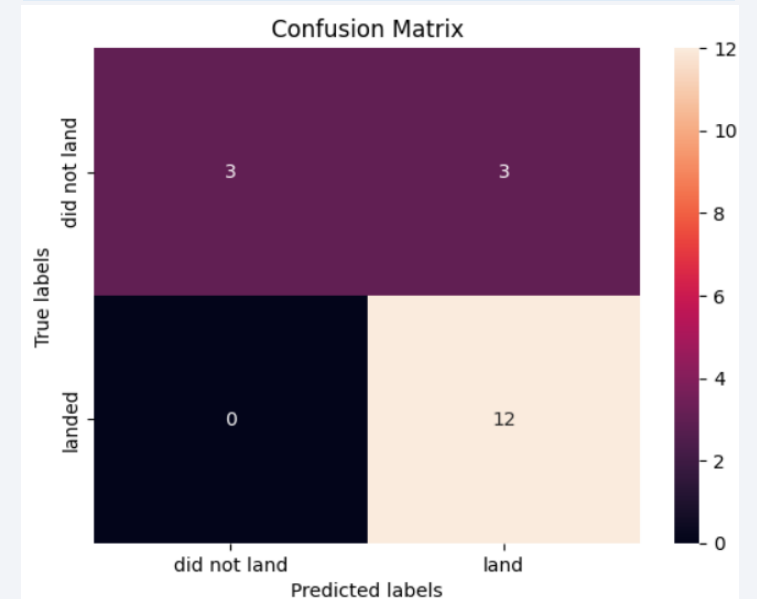
Confusion Matrix of Logistic Regression (LR)



Confusion Matrix of Support Vector Machine (SVR)



Confusion Matrix of K-Nearest Neighbors (KNN)



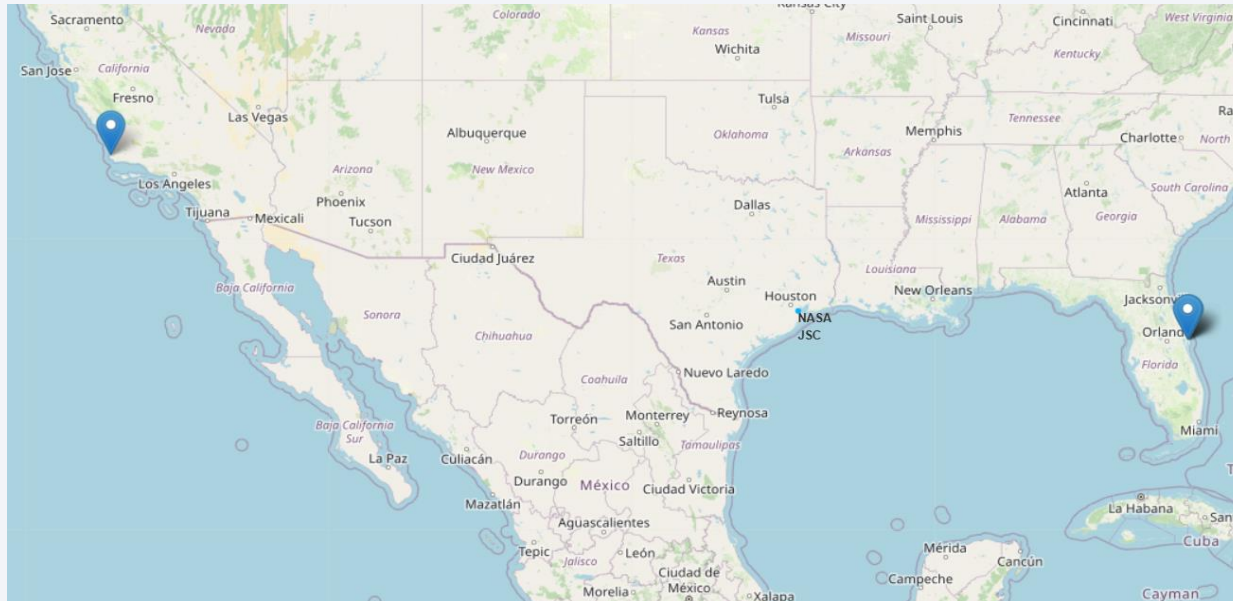
- LR, SVM, KNN models are the best performing. Their confusion matrices show that they predicted all the 12 successful landing correctly, although showing three false positives

Conclusions

- **LR, SVM, KNN** are the top-performing models for forecasting outcomes with this data
- Lighter payloads were more tested and generally showed higher performance compared to heavier ones.
- The likelihood of a SpaceX launch succeeding increases as the number of flights are progressing, suggesting a trend towards flawless launches over time.
- **Launch Complex 39A** at Kennedy Space Center has the highest number of successful launches compared to other launch sites.
- **ES-L1, GEO, SSO**, HEO orbit types exhibited the highest rates of successful landing. Interestingly, the first three of them displayed successful booster landing of light payloads
- "FT" booster version has a high success rate across various payload masses, demonstrating its reliability and robustness
- Interactive data visualizations using Folium and Plotly Dash provided valuable insights into the geographical and operational patterns of SpaceX launches, enabling stakeholders to make informed decisions based on comprehensive visual analytics



Appendix



- An intuitive way to locate sites on the map

```
#Locating the launch sites on the map
```

```
nasa_coordinates = [  
    [28.562302, -80.577356], [28.563197, -80.5  
]
```

```
for lat, lon in nasa_coordinates:  
    folium.Marker([lat, lon]).add_to(site_map)
```

```
site_map
```

Thank you!

